TWO POINTERS PATTERN

FAANG Master Guide — Java Edition

🎬 THE REAL-WORLD STORY

Imagine two people standing at opposite ends of a long hallway looking for a pair of numbers that add up to a target sum.

💡 THE TWO POINTERS AHA MOMENT

If the array is sorted, comparing `arr[left] + arr[right]` tells us EXACTLY which pointer to move! If sum is too small \rightarrow `left++`. If sum is too large \rightarrow `right--`. Replaces O(N^2) loops with **O(N) single pass**!

🧐 WHY NOT NESTED LOOPS?

- **Nested Loops O(N^2):** Checks every possible pair `(i, j)`. For N = 100,000, requires 10,000,000,000 operations \rightarrow TLE Error!
- **Two Pointers O(N):** Moves inward deterministically. Total steps <= N. Executes in 5 milliseconds!
- **Auxiliary Space O(1):** Reuses array pointers with 0 extra memory allocation.

📌 2 MAIN POINTER VARIATIONS

1. Opposite Direction (Converging):

```
left -> [ 1,  2,  4,  6,  8, 11 ] <- right
Sum = 1 + 11 = 12 > Target 10 -> right--
Sum = 1 + 8  = 9  < Target 10 -> left++
```

2. Same Direction (Fast & Slow / Partition):

```
[ slow, fast -> ] (In-place array modification)
```

📚 PREREQUISITES & COMPLEXITY REASONING

Problem Variation	Time Complexity	Auxiliary Space
Two Sum II (Sorted Array)	**O(N) single pass**	**O(1) in-place**
3Sum (Sort + 2-Pointer Loop)	**O(N^2)**	**O(1) space**
Container With Most Water	**O(N) single pass**	**O(1) space**
Trapping Rain Water (2 Pointers)	**O(N) single pass**	**O(1) space**

⚡ CLUES THAT SCREAM TWO POINTERS

1. *"Array is SORTED"* and asks for pair sum, triplet sum, or range search \rightarrow Opposite Two Pointers
2. *"Remove duplicates / zeros in-place in O(1) space"* \rightarrow Fast & Slow Pointers
3. *"Container with water / Trapping water at boundaries"* \rightarrow Converging Boundary Pointers

1 TEMPLATE A: OPPOSITE DIRECTION (CONVERGING)

```
public int[] twoSumSorted(int[] nums, int target) {
  int left = 0, right = nums.length - 1;
  while (left < right) {
    int sum = nums[left] + nums[right];
    if (sum == target) {
      return new int[]{left + 1, right + 1};
    } else if (sum < target) {
      left++;  // Needs larger value!
    } else {
      right--; // Needs smaller value!
    }
  }
  return new int[0];
}
```

2 TEMPLATE B: SAME DIRECTION (FAST & SLOW)

```
public int removeDuplicates(int[] nums) {
  if (nums.length == 0) return 0;
  int slow = 0;
  for (int fast = 1; fast < nums.length; fast++) {
    if (nums[fast] != nums[slow]) {
      slow++;
      nums[slow] = nums[fast]; // Overwrite in-place!
    }
  }
  return slow + 1; // Length of unique array!
}
```

TWO POINTERS PATTERN

Core Patterns — Part 1

PATTERN 1 VALID PALINDROME (Converging Alphanumeric Scan) e.g. Valid Palindrome (LC 125)

WHEN TO USE

Check symmetry from ends inward, skipping non-alphanumeric characters in O(N) time and O(1) space.

TEMPLATE — LC 125

```
public boolean isPalindrome(String s) {
  int l = 0, r = s.length() - 1;
  while (l < r) {
    while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
    while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
    if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt
(r))) {
      return false;
    }
    l++; r--;
  }
  return true;
}
```

PATTERN 2 TWO SUM II (Sorted Input Array) e.g. Two Sum II - Input Array Is Sorted (LC 167)

WHEN TO USE

Array is ALREADY sorted. Converge left/right pointers to find target in O(N) time and O(1) space without HashMap!

TEMPLATE — LC 167

```
public int[] twoSum(int[] numbers, int target) {
  int l = 0, r = numbers.length - 1;
  while (l < r) {
    int sum = numbers[l] + numbers[r];
    if (sum == target) return new int[]{l + 1, r + 1};
    if (sum < target) l++; else r--;
  }
  return new int[0];
}
```

PATTERN 3 3SUM (Sort + Fix Outer Pointer + 2-Pointer Loop) e.g. 3Sum (LC 15)

DUPLICATE SKIP REQUIREMENT

Sort array first. Loop `i` from 0 to N-3. Skip duplicates for `i`, `left`, and `right` to guarantee unique triplets in O(N^2) time!

TEMPLATE — LC 15

```
public List<List<Integer>> threeSum(int[] nums) {
  Arrays.sort(nums);
  List<List<Integer>> res = new ArrayList<>();
  for (int i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] == nums[i - 1]) continue; // Skip duplicate i!
    int l = i + 1, r = nums.length - 1;
    while (l < r) {
      int sum = nums[i] + nums[l] + nums[r];
      if (sum == 0) {
        res.add(Arrays.asList(nums[i], nums[l], nums[r]));
        while (l < r && nums[l] == nums[l + 1]) l++; // Skip duplicate
l!
        while (l < r && nums[r] == nums[r - 1]) r--; // Skip duplicate
r!
        l++; r--;
      } else if (sum < 0) l++; else r--;
    }
  }
  return res;
}
```

TWO POINTERS PATTERN

PATTERN 4 CONTAINER WITH MOST WATER (Greedy Boundary Shrink) e.g. Container With Most Water (LC 11)

GREEDY CHOICE PROOF

`Area = min(height[l], height[r]) * (r - l)`. The limiting factor is the shorter wall! Moving the taller wall CANNOT increase area because width shrinks and height is capped by shorter wall. Thus, ALWAYS move the shorter wall pointer!

TEMPLATE — LC 11

```
public int maxArea(int[] height) {
  int l = 0, r = height.length - 1;
  int maxWater = 0;
  while (l < r) {
    int h = Math.min(height[l], height[r]);
    maxWater = Math.max(maxWater, h * (r - l));
    if (height[l] < height[r]) {
      l++; // Move shorter wall!
    } else {
      r--; // Move shorter wall!
    }
  }
  return maxWater;
}
```

PATTERN 5 TRAPPING RAIN WATER (2-Pointer Maximum Tracking) e.g. Trapping Rain Water (LC 42)

WATER TRAP MECHANICS

Maintain `leftMax` and `rightMax`. If `leftMax < rightMax`, trapped water at `left` is determined solely by `leftMax - height[left]`! Runs in O(N) time and O(1) space.

TEMPLATE — LC 42

```
public int trap(int[] height) {
  int l = 0, r = height.length - 1;
  int leftMax = 0, rightMax = 0, water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] >= leftMax) leftMax = height[l];
      else water += leftMax - height[l];
      l++;
    } else {
      if (height[r] >= rightMax) rightMax = height[r];
      else water += rightMax - height[r];
      r--;
    }
  }
  return water;
}
```

TWO POINTERS PATTERN

Decision Tree & Trigger Words

TWO POINTERS — WHICH VARIATION TO USE?

Start: What is the input array type & goal?

- 🎯 **Sorted array pair/triplet sum** → **Opposite Pointers** `left = 0, right = n - 1`
- ✂️ **In-place duplicate removal / partition** → **Fast & Slow Pointers** `slow` tracks valid write idx
- 🌊 **Max area / Trapping water** → **Boundary Max Converge** Move shorter boundary wall
- 🔤 **String symmetry / Palindrome** → **Alphanumeric Converge** Skip non-char, match ends

🎯 TRIGGER WORDS DIRECTORY

Trigger Word	Variant	Action
"Sorted array, find pair sum"	Opposite Two Pointers	`l++` if sum small, `r--` if large
"3Sum / 4Sum"	Sort + Outer Loop + 2-Pointer	Skip duplicate values on all 3 pointers
"Container with most water"	Shorter Wall Move	Move `l` if `h[l] < h[r]` else `r`
"Trapping rain water"	2-Pointer Boundary Max	Compare `leftMax` vs `rightMax`

💼 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why sorting for 3Sum if array is unsorted?	Sorting takes O(N log N) time, making the overall O(N^2) 2-pointer scan possible while avoiding complex hash set duplicate checks!
How to prevent duplicate triplets in 3Sum?	Skip duplicate values for `nums[i]`, `nums[l]`, and `nums[r]` using `while` loops after finding a valid sum!

EASY — Basic Converging & Partition

LC 125 · Valid Palindrome — EASY

Pattern: Converging Pointers
Key Insight: Skip non-alphanumeric chars.

```
public boolean isPalindrome(String s) {
  int l = 0, r = s.length() - 1;
  while (l < r) {
    while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
    while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
    if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r))) return false;
    l++; r--;
  }
  return true;
}
```

LC 167 · Two Sum II — EASY

Pattern: Sorted Converging
Key Insight: `l++` if sum small, `r--` if large.

```
public int[] twoSum(int[] numbers, int target) {
  int l = 0, r = numbers.length - 1;
  while (l < r) {
    int sum = numbers[l] + numbers[r];
    if (sum == target) return new int[]{l + 1, r + 1};
    if (sum < target) l++; else r--;
  }
  return new int[0];
}
```

LC 26 · Remove Duplicates — EASY

Pattern: Fast & Slow Pointers
Key Insight: Overwrite `nums[++slow] = nums[fast]`.

```
public int removeDuplicates(int[] nums) {
  if (nums.length == 0) return 0;
  int slow = 0;
  for (int fast = 1; fast < nums.length; fast++) {
    if (nums[fast] != nums[slow]) { slow++; nums[slow] = nums[fast]; }
  }
  return slow + 1;
}
```

MEDIUM — Triplets & Water Containers

LC 15 · 3Sum — MEDIUM

Pattern: Sort + 2 Pointers
Key Insight: Fix `i`, converge `l` & `r`, skip dups.

```
public List<List<Integer>> threeSum(int[] nums) {
  Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>();
  for (int i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] == nums[i - 1]) continue;
    int l = i + 1, r = nums.length - 1;
    while (l < r) {
      int sum = nums[i] + nums[l] + nums[r];
      if (sum == 0) {
        res.add(Arrays.asList(nums[i], nums[l], nums[r]));
        while (l < r && nums[l] == nums[l + 1]) l++;
        while (l < r && nums[r] == nums[r - 1]) r--;
        l++; r--;
      } else if (sum < 0) l++; else r--;
    }
  }
  return res;
}
```

LC 11 · Container With Most Water — MEDIUM

Pattern: Shorter Wall Converge
Key Insight: Always move pointer of shorter wall.

```
public int maxArea(int[] height) {
  int l = 0, r = height.length - 1, maxW = 0;
  while (l < r) {
    int h = Math.min(height[l], height[r]);
    maxW = Math.max(maxW, h * (r - l));
    if (height[l] < height[r]) l++; else r--;
  }
  return maxW;
}
```

HARD — Trapping Rain Water

LC 42 · Trapping Rain Water

HARD

Pattern: Boundary Max Tracking

Key Insight: Trapped water bounded by `leftMax` & `rightMax`.

```
public int trap(int[] height) {
  int l = 0, r = height.length - 1;
  int lMax = 0, rMax = 0, water = 0;
  while (l < r) {
    if (height[l] < height[r]) {
      if (height[l] >= lMax) lMax = height
[l];
      else water += lMax - height[l];
      l++;
    } else {
      if (height[r] >= rMax) rMax = height
[r];
      else water += rMax - height[r];
      r--;
    }
  }
  return water;
}
```

MEDIUM — 3Sum Closest

LC 16 · 3Sum Closest

MEDIUM

Pattern: Sort + Min Diff Converge

Key Insight: Track minimum absolute difference.

```
public int threeSumClosest(int[] nums, int t
arget) {
  Arrays.sort(nums);
  int closest = nums[0] + nums[1] + nums[2];
  for (int i = 0; i < nums.length - 2; i++)
{
    int l = i + 1, r = nums.length - 1;
    while (l < r) {
      int sum = nums[i] + nums[l] + nums[r];
      if (Math.abs(target - sum) < Math.abs
(target - closest)) closest = sum;
      if (sum < target) l++; else r--;
    }
  }
  return closest;
}
```

COMPLETE TWO POINTERS PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff
125	Valid Palindrome	Converging Alphanumeric Scan	O(N)	O(1)	E
167	Two Sum II	Sorted Converging Pointers	O(N)	O(1)	E
26	Remove Duplicates	Fast & Slow Pointers	O(N)	O(1)	E
15	3Sum	Sort + Outer Loop + 2-Pointer	O(N^2)	O(1)	M
11	Container With Most Water	Shorter Wall Converge	O(N)	O(1)	M
16	3Sum Closest	Sort + Min Difference Check	O(N^2)	O(1)	M
42	Trapping Rain Water	2-Pointer Boundary Max Tracking	O(N)	O(1)	H

🔍 DRY RUN — Container With Most Water ([1, 8, 6, 2, 5, 4, 8, 3, 7])

Step	l (val)	r (val)	width (r - l)	h = min(h[l], h[r])	area	maxArea	Action
1	0 (1)	8 (7)	8	min(1, 7) = 1	8	8	`l++` (1 < 7)
2	1 (8)	8 (7)	7	min(8, 7) = 7	49	49	`r--` (7 < 8)
3	1 (8)	7 (3)	6	min(8, 3) = 3	18	49	`r--` (3 < 8)
4	1 (8)	6 (8)	5	min(8, 8) = 8	40	49 ✅	`r--`

📐 MATHEMATICAL PROOF — GREEDY CHOICE OF SHORTER WALL

Claim: In Container With Most Water, moving the taller wall pointer cannot produce a larger area.

Proof: Area is defined as `min(h[l], h[r]) * width`. If `h[l] < h[r]`, max height is capped at `h[l]`. Moving `r` inward decreases `width` while height remains at most `h[l]`. Thus, area is strictly smaller! Moving `l` is the ONLY way to potentially find a taller wall \rightarrow **Exact Proof of Correctness!**

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

☐ Code Valid Palindrome with alphanumeric skip
☐ Code Two Sum II on sorted array in O(N)
☐ Code Remove Duplicates in O(1) space
☐ Implement 3Sum with triply nested duplicate skip
☐ Code Container With Most Water in 3 minutes
☐ Implement Trapping Rain Water with 2-pointers
☐ Explain greedy choice proof for water container
☐ Differentiate Converging vs Fast/Slow pointers

💼 GOLDEN RULES — NEVER FORGET

Rule 1 — Always Sort First for Sum Problems

Before using Two Pointers for Two Sum / 3Sum / 4Sum, ensure the array is sorted (`Arrays.sort(nums)`). The entire directional choice relies on sorted order!

Rule 2 — Guard Against Out-of-Bounds in Duplicate Skips

When skipping duplicates inside `while` loops (e.g. in 3Sum), ALWAYS include `while (l < r && nums[l] == nums[l + 1]) l++` to prevent index out of bounds errors!